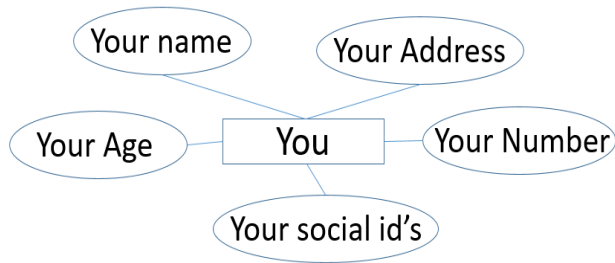


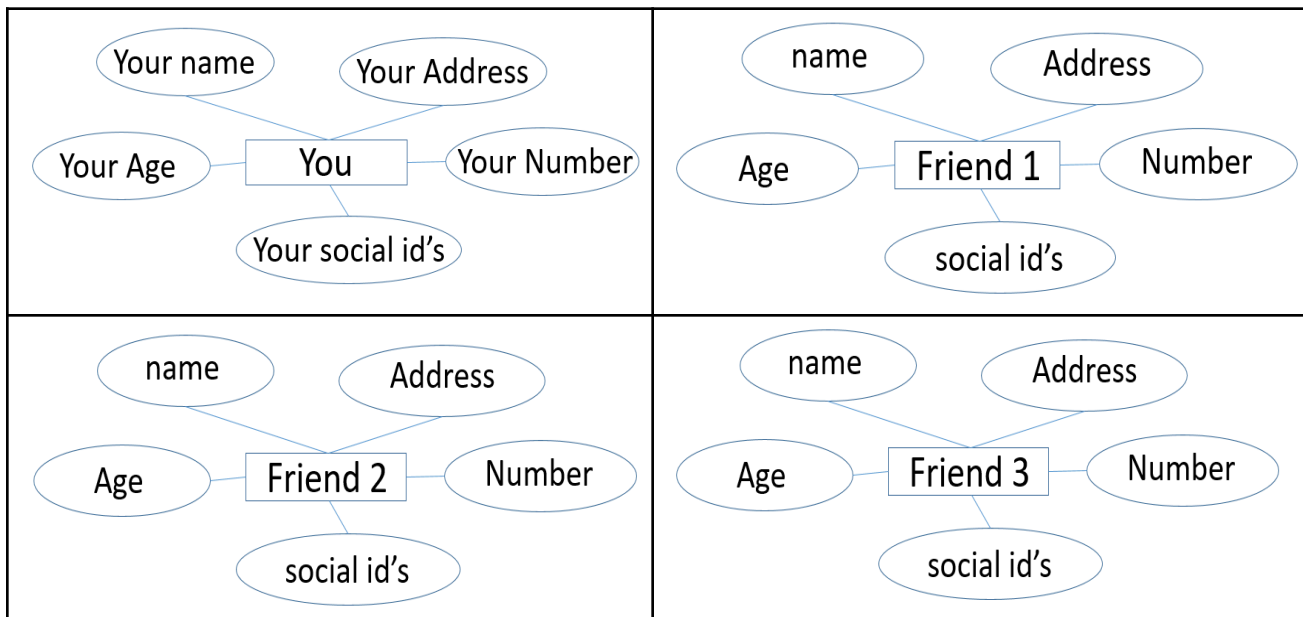
## **DBMS Introduction, Definition, Example, Types**

Data: Any information, facts-figures related to any object, known as data.

Database is a collection of different types of relevant or irrelevant data. Example: Mark sheet of a student, Google.



Database Management System is a collection of database and some computer programs which are used to manage the database. Example: Online result system.



DBMS Administrator: That person or authority which manage or operate the database.

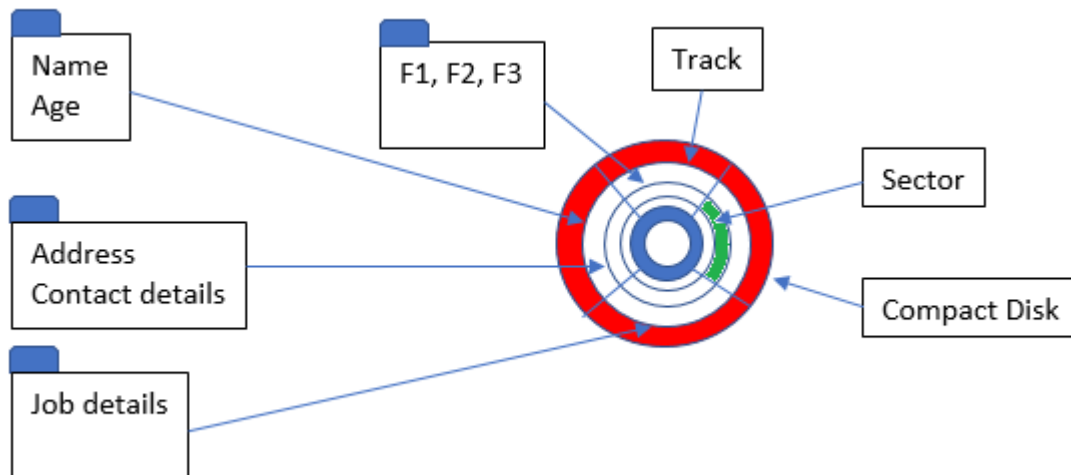
Why DBMS: Easy to access, manage, operate, store large amount of data.

Types of DBMS:

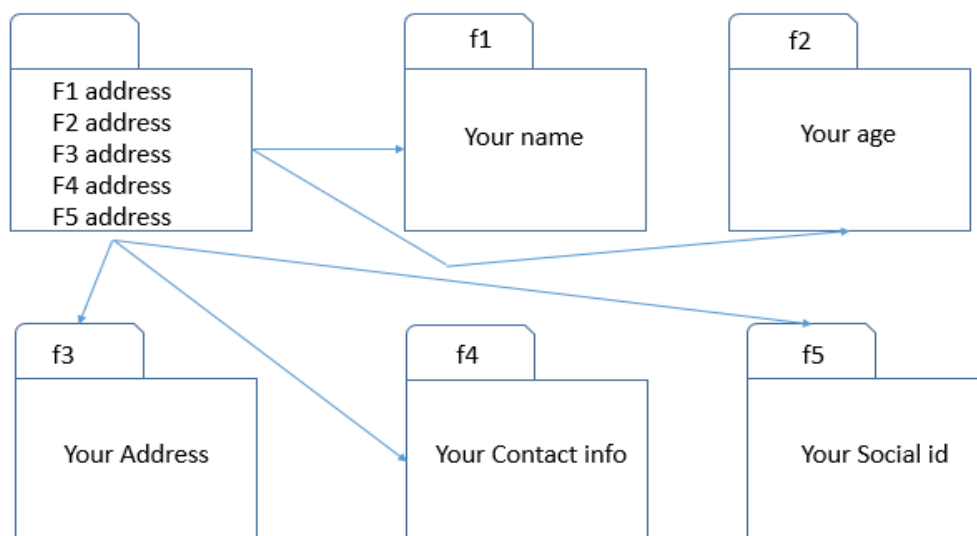
1. Single or Flat File database
2. Multi File, Relational or Structured Database
  - 2.1) Relational DBMS
  - 2.2) Hierarchical DBMS
  - 2.3) Network DBMS
  - 2.4) Object- Oriented DBMS

## File System

In the 1960's, flat file system was introduced. Computer stores data in different sizes of blocks in its memory. This is known as file. In file system data is stored in these files. These files have limited capacity, so we need to create lots of files to store big size data. This kind of data storage is known as file system.



If we want to store data of yours, then one file is created with your name, in which your basic details could be stored, but these kinds of files have very limited storage space, so we need to create lots of files to store your whole data. To access these all files, we need to give these all files a unique number or name. then we need to create a file which contains the numbers or names of all remaining files. It's like an index of a book. This system is very lengthy and we need to learn computer language programming commands to manage this system.



Advantages:

- 1) Due to distributed files virus issue is not such a big concern.
- 2) This was such a day saver system in the early time period of computers.

Disadvantages:

- 1) Very complex structure.

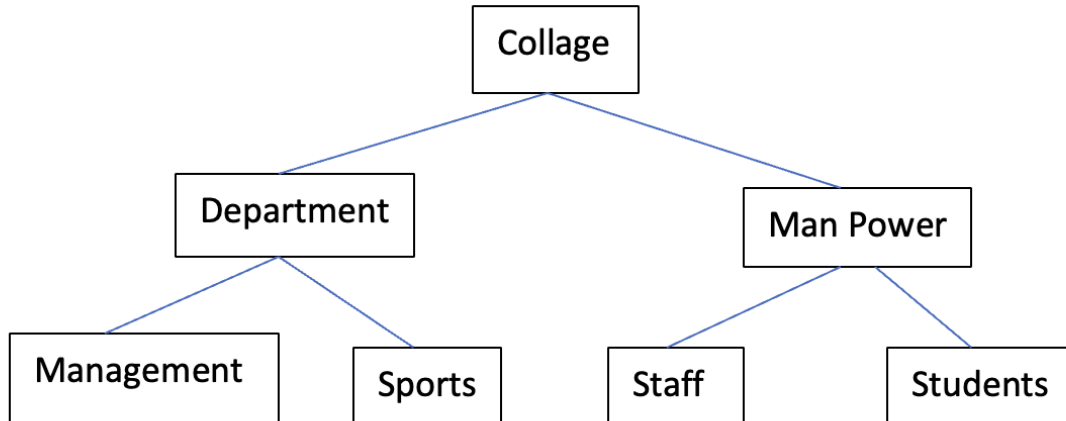
- 2) Due to limited storage capacity we need to create lots of files to store even a single object's data.
- 3) If index file is lost, then all related files are useless. We need to create them all again. This creates lots of copies of the same data.
- 4) Any types of changes in any file leads to create whole new set of files again.
- 5) We need to know the programming languages.
- 6) This system is based on navigation technique, so it consumes lots of time and efforts.
- 7) In this system lots of memory space is wasted.

Instead of this all file system was a revolutionary invention which plot the seeds of modern database. File system is still very useful. Database can't be used everywhere. Like if we want to install an application or a software, then we need to download its files first. Then these files run the commands on the system, afterwards the application or software will run on our system.

In short if the whole data contained in a file needs to be used at once, then file system is best for that, like installation files.

## **Hierarchical Data Model**

In this model, a tree structure is followed to store data. Each data field is known as Node. In this model step by step navigation is performed.



Here collage node is a parent node, and Department, man power nodes are child nodes of parent node. Similarly, management and sports are child node of Department. So at a same time a node can play both roles parent and child. The top most node is known as Root Node.

Advantages:

Up Side Down approach followed.

Parent Child terminology applied.

1 to many relationships is good to remove redundancy.

Disadvantages:

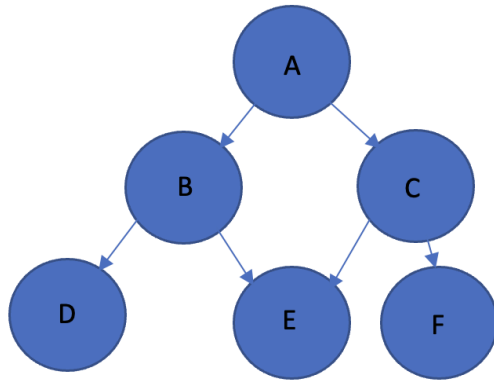
It is little bit complex.

Many to Many relationships is not allowed, so navigation take more time.

Remote entry is not allowed. We need to go through the root node and parent node to reach any specific child node.

## Network Data Model

It is same as hierarchical model, but in this child node can have more than one parent. Relation is shown by Arrows. It follows navigation or indexing technique.



### Advantages:

1. More flexible than hierarchical model.
2. Direct approach is allowed.
3. 1 to 1, 1 to many relationships is allowed.

### Disadvantages:

1. It's more complex.
2. Slow and difficult to maintain and operate.
3. Diagram structures are very complex and massive.
4. If we update even a single node, the whole dependency changed.

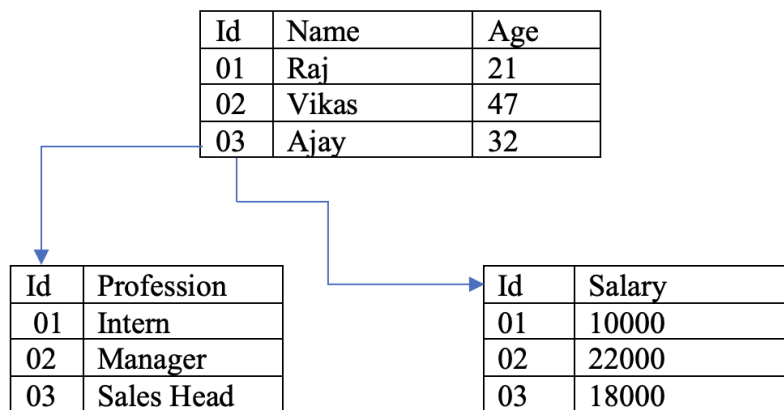
## Relational DBMS

In this tables are used to store data. Tables show relation between data contained. That's why these are called table or relation. Columns are used to store fields or attributes, and rows are used to store data of the Entity. (Entity are real world objects for computers like, mobile, human, vehicles are Entity for computer.)

In this we can splits large data into smaller and simpler tables and connect them all with the help of unique key concept.

Each block of the table is called Tuple or record.

SQL (Structured Query Language) is used in this DBMS.



Advantages:

1. Easy to create, maintain, operate.
2. Very flexible.
3. Multiple tables allowed, which improves data simplification.
4. Single tuple operation is allowed, means the changed will effect only the selected tuple, remaining table will be remained same.
5. Unique key concept like Primary key, foreign key makes operations easier and safe.

Disadvantages:

1. Software used to create these are very expensive, and this kind of software requires expensive hardware as well.
2. Highly skilled and creative man power is required.

## File System vs DBMS

|                       | File System                                                                                                                                                                           | DBMS                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Programming Knowledge | Need to learn how to use command prompt, how to apply commands, No Graphics buttons only text.                                                                                        | Graphical User Interface, visual buttons given, no programming knowledge required.                         |
| View                  | All details are visible to everyone.                                                                                                                                                  | Abstract View, means only required information is visible.                                                 |
| Data Recovery         | No option of recovery.                                                                                                                                                                | Easily recoverable, but permission required.                                                               |
| Security              | No Security, if you know the password of file, you can see all contents of the file. Anyone can hack.                                                                                 | Even you know password, you can see only pre allowed range of data. Can't be hacked easily.                |
| Redundancy            | Data copied so many times. Even on update, modify a single data, a new file was created every time and the old one also exists. Consumed lots of space and data inconsistency.        | No redundancy. Every time the original data field or record is updated.                                    |
| Functioning           | Only storage, only manual operations allowed. Like passing commands.                                                                                                                  | Lots of automatic functions and operations allowed. Like visual click buttons instead of passing commands. |
| Data Consistency      | Weak due to redundancy. For a single object multiple files have different data.                                                                                                       | Strong, because only one record exists.                                                                    |
| Data Integrity        | (Alteration in data create current file different from the original source file.)<br>Weak due to redundancy. Due to multiple files for a single object no actual source is confirmed. | Strong. Due to strong Consistency and no redundancy.                                                       |
| Data Representation   | Weak, only black or blue background with plain text, no GUI.                                                                                                                          | Very attractive, GUI based environment and functions available.                                            |
| Cost                  | Low, most of time almost free.                                                                                                                                                        | Very costly, due to costly software and Hardware.                                                          |
| Concurrent access     | Due to redundancy accurate concurrent access is not possible, because every user uses a completely new or different file.                                                             | Due to no redundancy and centralized structure, actual and accurate concurrent access allowed.             |

Hence file system is still existing, because its working style. At those places where all data present needs to be executed without any choice, the file system is required. Like the setup files or boot files for any application which we want to install, we need to download and execute all the files of that application.

On the other hand database is used at those places where only selected data is required at once.

## RELATIONAL DATA MODEL

Relational data models are used to store data in **tabular** form. Data can be stored in multiple tables and tables can be related to each other. Even in a single table all column and rows are logically interconnected. In computer language table word is used for represent relations between data.

| Name | Age | Address | Email | Mobile | Department | Role | Salary |
|------|-----|---------|-------|--------|------------|------|--------|
|      |     |         |       |        |            |      |        |
|      |     |         |       |        |            |      |        |
|      |     |         |       |        |            |      |        |

**Table “Company Employee Details”**

Here in this table each column is a **field** or an attribute. Every row is an **information set or data set** for individual entity. Every cell or block of the table is known as **tuple**.

Every field has its own unique data type. Like in name only alphabets are allowed, in age only numeric values are allowed. In address both alphabets and numbers are allowed.

In this table we have 3 blank rows, it means we can store 3 person’s information in this table. This is just an example. The actual tables are very bigger and complex in size.

In relational database bigger tables are split in smaller tables to make computing easy. These small split tables shown as a single big table.

In table each data set must be unique from each other, that’s why we must add an unique attribute to each data set, like registration number or ID number, which is unique for every data set.

| ID | Name | Age | Address | Email | mobile | Department | Role | Salary |
|----|------|-----|---------|-------|--------|------------|------|--------|
|    |      |     |         |       |        |            |      |        |
|    |      |     |         |       |        |            |      |        |
|    |      |     |         |       |        |            |      |        |

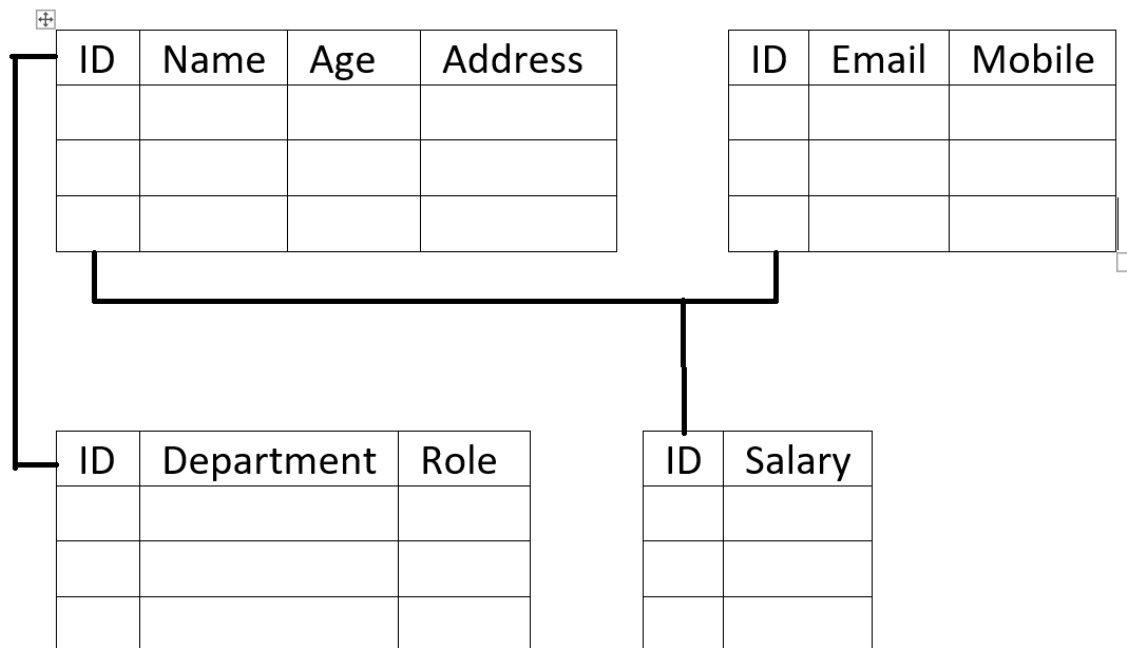
| ID | Name | Age | Address |
|----|------|-----|---------|
|    |      |     |         |
|    |      |     |         |
|    |      |     |         |

| ID | Email | Mobile |
|----|-------|--------|
|    |       |        |
|    |       |        |
|    |       |        |

| ID | Department | Role |
|----|------------|------|
|    |            |      |
|    |            |      |
|    |            |      |

| ID | Salary |
|----|--------|
|    |        |
|    |        |
|    |        |

**Split tables for Table “Employee”**



**Connection between tables**

## **KEY CONCEPTS**

Key is unique for every lock. Similarly, those attributes which are unique for every entity in a single table, known as key attributes.

Keys are nothing else only the unique attributes associated to every entity.

Here are main keys which we can use in a table:

1. **Candidate**
2. **Primary**
3. **Alternate**
4. **Foreign**
5. **Composite**
6. **Super**
7. **Unique**

**1. Candidate key:** Those attributes which may be unique for an entity, can be considered as Candidate keys, like ID, name, mobile number, address in the above given table.

**2. Primary keys:** From candidate keys we choose a key which is almost unique for every entity, called Primary key for that entity and the whole table. Name, address, mobile number may be not unique, but ID number will be unique for every entity, so we choose ID number as a primary key.

The primary keys should be easy to find, allocate, and not be much complex.

Like mobile numbers are also unique for every entity, but mobile number contains at least 12-digit string, which is quite complex and lengthy for rapidly use, so we can't use it as Primary key. Instead of this the ID number may be shorter and follow a fix

sequence, easy to generate and allocate, that's why we should use ID number as a Primary key.

3. **Alternate keys:** After selecting our primary key, remaining candidate keys are considered as Alternate keys. Which may be used at the place of primary key in any specific situation.
4. **Foreign Key:** When we use primary key to join tables, then we need to put the primary key in every table. In the main table it is considered as Primary key but in other tables it is considered as foreign key. Like in the above given table "Employee", the ID number is Primary key, but if we use it in any other table of the database, then it is considered as foreign key in others tables, and those other tables also have their own primary key as well. Primary key and foreign key concept is used to connect tables.
5. **Composite key:** When we merge two or more candidate keys to create more powerful or unique key for any entity or table then it is known as composite key, but the primary key should be part of the composite key. Sometimes we can use any ordinary attribute also to merge with primary key to create a composite key.
6. **Super key:** Sometimes only one primary key is not enough to maintain uniqueness for an entity, in that case we merge primary key with any one or more than one alternate key to create a module of primary key, then this is known as super key. Like "ID + Email = Super key".  
In this case for every entity ID and Email both the attributes must be unique. If anyone one of them may not be unique then the entity will not be taken in the table.
7. **Unique key:** Primary keys never accept NULL values. So, if we want to make any attribute a primary key with NULL values then this key will call Unique key. Most of times Unique key is created by merge two or more alternate keys. In which Primary key should not be null but the other alternate key may contain Null Value. Like "ID + Email = Unique key". In this the ID is primary key and can't allow Null Value, but the Email may hold Null value.

**NULL:** This is a keyword, which shows the table that in this tuple or block no value is given, so at run time show this block as blank. We can't leave any tuple or block blank, otherwise the whole data set will be rejected, so at the blank place we may put NULL.

| Id | Name | Age  | Position |
|----|------|------|----------|
| 01 | Ajay | 23   | HR       |
| 02 | Raj  | NULL | Manager  |

When we see the result then it will look like this:

| Id | Name | Age | Position |
|----|------|-----|----------|
| 01 | Ajay | 23  | HR       |
| 02 | Raj  |     | Manager  |